Rekurencyjne algorytmy mnożenia macierzy

Mateusz Majta, Maksymilian Siemek

Algorytmy Macierzowe 2026/2027, grupa 2

Spis treści

1 Wstęp **2**

2 Specyfikacja narzędzi i sprzętu **2**

3 Algorytmy **2**

3.1 Modyfikacja algorytmów do mnożenia macierzy o dowolnych rozmiarach 2

3.2 Metoda Binét'a 4

3.3 Metoda Strassena 5

3.4 Metoda Ladermana 6

4 Porównanie algorytmów **7**

4.1 Liczba operacji 7

4.2 Czas działania 8

4.3 Oszacowanie złożoności obliczeniowej 8

4.4 Weryfikacja poprawności: przykład 4×4 9

1 Wstęp

Celem ćwiczenia była implementacja i porównanie trzech rekurencyjnych algorytmów mnożenia macierzy: metody Binét'a, metody Strassena [1] oraz metody Ladermana. Wszystkie trzy mnożą macierze kwadratowe $n \times n$ o *dowolnym* rozmiarze (nie tylko będącym potęgą dwójki lub trójki), a macierze zawierają liczby losowe z rozkładu równomiernego na przedziale $[10^{-8}, 1)$. Razem ze sprawozdaniem zawarty jest kod w języku C++ zawierający implementację wszystkich algorytmów (`binet.cpp`, `strassen.cpp`, `laderman.cpp`) oraz skrypty służące do generowania wykresów (`make_plots.py`) i do porównania wyników z Octave/MATLAB (`compare.m`).

2 Specyfikacja narzędzi i sprzętu

Język nie jest interpretowany: C++ jest kompilowany z góry do natywnego kodu maszynowego ARM64 (Apple clang 21.0.0 wywoływany jako `g++`, flagi `-O2 -std=c++17`); powstały plik wykonywalny jest uruchamiany bezpośrednio przez procesor, bez interpretera i maszyny wirtualnej. Obliczenia były jednowątkowe. Obliczenia zostały dokonane na komputerze z procesorem Apple M1 Pro (8 rdzeni: 6 wydajnych i 2 energooszczędne), pamięcią RAM 16 GB (LPDDR5) i systemem macOS 26.5.

Czas mierzono zegarem `std::chrono::steady_clock` tylko dla samego mnożenia (bez generowania macierzy i weryfikacji): dla $n \leq 200$ jako minimum z 3 powtórzeń, dla $n > 200$ jako pojedynczy pomiar. Wszystkie trzy programy uruchamiano kolejno, na tym samym komputerze, więc ich czasy są ze sobą porównywalne. Pomiary czasu wykonano dla $n = 1, 2, \ldots, 200$ oraz $n = 250, 300, \ldots, 1000$ (krok 50); liczbę operacji wyznaczono dla *każdego* $n = 1, \ldots, 1000$, korzystając z modelu rekurencyjnego (to samo drzewo wywołań co w algorytmie, ale bez mnożenia macierzy). Dla wszystkich zmierzonych n model daje liczbę operacji identyczną z licznikiem w działającym programie. Maksymalny rozmiar, który udało się policzyć wszystkimi trzema metodami: $n = 1000$.

3 Algorytmy

Omawiane algorytmy polegają na zasadzie „dziel i rządź", rekurencyjnie dzieląc macierze na podmacierze nazywane inaczej blokami. W klasycznej postaci przyjmują one ustalone rozmiary macierzy (parzyste dla metod Binét'a i Strassena, podzielne przez 3 dla metody Ladermana), co sprawia, że użycie ich w przypadkach ogólnych wymaga wprowadzenia kilku modyfikacji.

3.1 Modyfikacja algorytmów do mnożenia macierzy o dowolnych rozmiarach

Problem arbitralnych rozmiarów macierzy rozwiązaliśmy bez użycia tzw. „paddingu" (wypełniania macierzy zerami), dzięki czemu liczba operacji nie jest sztucznie zawyżona. W metodzie Binét'a przy nieparzystych wymiarach bloki górne/lewe mają $\lceil \cdot/2 \rceil$, a dolne/prawe $\lfloor \cdot/2 \rfloor$ wierszy i kolumn, więc mnożymy macierze prostokątne $(p \times q) \cdot (q \times r)$, a wzory na bloki wyniku pozostają poprawne. W metodzie Strassena dla nieparzystego n „obieramy" ostatni wiersz i kolumnę (*dynamic peeling* [2]): rdzeń $(n-1) \times (n-1)$ mnożymy rekurencyjnie, a pozostałe, cienkie iloczyny klasycznie. W metodzie Ladermana stosujemy to samo obieranie, tylko odcinamy $r = n \bmod 3 \in \{1, 2\}$ wierszy i kolumn, tak aby rdzeń $(n-r) \times (n-r)$ dzielił się przez 3. Rekurencję prowadzimy do najmniejszych możliwych bloków (bez progu przełączenia na metodę klasyczną): 1×1 u Binét'a i Strassena oraz 1×1 lub 2×2 u Ladermana. Na Rysunkach 1, 2, 3 znajdują się najważniejsze fragmenty kodu w języku C++ rozwiązujące te przypadki, a na Rysunku 4 sposób zliczania operacji zmiennoprzecinkowych.

```
Matrix multiplyBinet(const Matrix& A, const Matrix& B) {
    const int p = A.rows, q = A.cols, r = B.cols;
    if (p == 1 || q == 1 || r == 1) return multiplyNaive(A, B);   // przypadek bazowy
    const int ph = (p + 1) / 2, qh = (q + 1) / 2, rh = (r + 1) / 2;  // punkty podzialu
    Matrix A11 = subMatrix(A, 0, 0, ph, qh), A12 = subMatrix(A, 0, qh, ph, q - qh);
    Matrix A21 = subMatrix(A, ph, 0, p - ph, qh), A22 = subMatrix(A, ph, qh, p - ph, q - qh);
    // ... B11, B12, B21, B22 wycinane analogicznie (wiersze wg qh, kolumny wg rh)
    Matrix C11 = add(multiplyBinet(A11, B11), multiplyBinet(A12, B21));   // 8 mnozen
    Matrix C12 = add(multiplyBinet(A11, B12), multiplyBinet(A12, B22));
    Matrix C21 = add(multiplyBinet(A21, B11), multiplyBinet(A22, B21));
    Matrix C22 = add(multiplyBinet(A21, B12), multiplyBinet(A22, B22));
    // ... sklejenie C11..C22 w macierz wynikowa p x r (placeBlock)
}
```

Rysunek 1: Arbitralne rozmiary macierzy, metoda Binét'a (`binet.cpp`, fragment skrócony)

```
Matrix multiplyStrassen(const Matrix& A, const Matrix& B) {
    const int n = A.rows;
    if (n <= STRASSEN_CUTOFF) return multiplyNaive(A, B);   // CUTOFF = 1
    if (n % 2 == 1) {                                        // obieranie: m = n-1 parzyste
        // ... A11 (m x m), a12, a21, a22 oraz b.. wycinane przez subMatrix
        Matrix C11 = add(multiplyStrassen(A11, B11), multiplyNaive(a12, b21));
        // ... C12, C21, C22: tylko "cienkie" iloczyny multiplyNaive + add
    }
    // n parzyste: 7 mnozen rekurencyjnych zamiast 8
    Matrix M1 = multiplyStrassen(add(A11, A22), add(B11, B22));
    Matrix M2 = multiplyStrassen(add(A21, A22), B11);
    Matrix M3 = multiplyStrassen(A11, sub(B12, B22));
    Matrix M4 = multiplyStrassen(A22, sub(B21, B11));
    Matrix M5 = multiplyStrassen(add(A11, A12), B22);
    Matrix M6 = multiplyStrassen(sub(A21, A11), add(B11, B12));
    Matrix M7 = multiplyStrassen(sub(A12, A22), add(B21, B22));
    Matrix C11 = add(sub(add(M1, M4), M5), M7);   Matrix C12 = add(M3, M5);
    Matrix C21 = add(M2, M4);                     Matrix C22 = add(add(sub(M1, M2), M3), M6);
    // ... sklejenie C11..C22 w macierz wynikowa n x n
}
```

Rysunek 2: Arbitralne rozmiary macierzy, metoda Strassena (`strassen.cpp`, fragment skrócony)

```
Matrix combine(MatrixList plus, MatrixList minus = {}) {  // plus[0] + plus[1] + ... - minus[0] - ...
    auto it = plus.begin();
    Matrix C = *it;                                       // kopia pierwszego skladnika (bez operacji)
    for (++it; it != plus.end(); ++it) { /* C += *it */ g_flops += C.data.size(); }
    for (const Matrix& X : minus)      { /* C -= X   */ g_flops += C.data.size(); }
    return C;
}
Matrix multiplyLaderman(const Matrix& A, const Matrix& B) {
    const int n = A.rows;
    if (n <= LADERMAN_CUTOFF) return multiplyNaive(A, B);    // CUTOFF = 2
    const int r = n % 3;
    if (r != 0) { /* obieranie r wierszy i kolumn - jak u Strassena, rdzen m = n - r */ }
    const int h = n / 3;
    // ... A11..A33, B11..B33 wycinane przez subMatrix (bloki h x h)
    Matrix M1 = multiplyLaderman(combine({A11, A12, A13}, {A21, A22, A32, A33}), B22);
    Matrix M2 = multiplyLaderman(combine({A11}, {A21}), combine({B22}, {B12}));
    // ... M3..M23 analogicznie, wg wzorow (2)
    Matrix C11 = combine({M6, M14, M19});
    Matrix C12 = combine({M1, M4, M5, M6, M12, M14, M15});
    // ... C13..C33 wg wzorow (3), sklejenie 9 blokow w macierz n x n
}
```

Rysunek 3: Arbitralne rozmiary macierzy, metoda Ladermana (`laderman.cpp`, fragment skrócony). Sumy mają do 7 bloków, dlatego funkcja `combine` przyjmuje listę bloków dodawanych i odejmowanych

```cpp
static unsigned long long g_flops = 0;           // licznik operacji + - * /
Matrix add(const Matrix& A, const Matrix& B) {   // sub() analogicznie
    Matrix C(A.rows, A.cols);
    for (size_t k = 0; k < C.data.size(); ++k) C.data[k] = A.data[k] + B.data[k];
    g_flops += C.data.size();                    // 1 operacja na element
    return C;
}
```

Rysunek 4: Zliczanie operacji zmiennoprzecinkowych. Licznik rośnie tylko tutaj oraz w mnożeniu klasycznym (przypadek bazowy: pqr mnożeń i $pr(q-1)$ dodawań); `combine` liczy się tak samo

3.2 Metoda Binét'a

Mnożenie macierzy A i B metodą Binét'a odpowiada „naiwnemu" sposobowi mnożenia macierzy. Polega ona na bezpośrednim pomnożeniu każdego wiersza bloków macierzy A z każdą kolumną bloków macierzy B (1):

$$C = AB = \begin{bmatrix} \begin{bmatrix} A_{11} & A_{12} \end{bmatrix} \begin{bmatrix} B_{11} \\ B_{21} \end{bmatrix} & \begin{bmatrix} A_{11} & A_{12} \end{bmatrix} \begin{bmatrix} B_{12} \\ B_{22} \end{bmatrix} \\ \begin{bmatrix} A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} \\ B_{21} \end{bmatrix} & \begin{bmatrix} A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{12} \\ B_{22} \end{bmatrix} \end{bmatrix} \tag{1}$$

Algorytm ten zapisany w postaci rekurencyjnej procedury wygląda następująco (Algorytm 1):

Algorytm 1 Rekurencyjne mnożenie macierzy metodą Binét'a

Procedura Pomnóż(A, B) ▷ *A: $p \times q$, B: $q \times r$*
  Jeżeli $p = 1$ lub $q = 1$ lub $r = 1$ **wykonaj**
    Zwróć MnożenieKlasyczne(A, B)
  $p_h, q_h, r_h \leftarrow \lceil p/2 \rceil, \lceil q/2 \rceil, \lceil r/2 \rceil$ ▷ *punkty podziału*
  podziel A w punktach (p_h, q_h), a B w punktach (q_h, r_h) na bloki $A[i][j]$, $B[i][j]$
  $C[1][1] \leftarrow$ Pomnóż$(A[1][1], B[1][1])$ + Pomnóż$(A[1][2], B[2][1])$
  $C[1][2] \leftarrow$ Pomnóż$(A[1][1], B[1][2])$ + Pomnóż$(A[1][2], B[2][2])$
  $C[2][1] \leftarrow$ Pomnóż$(A[2][1], B[1][1])$ + Pomnóż$(A[2][2], B[2][1])$
  $C[2][2] \leftarrow$ Pomnóż$(A[2][1], B[1][2])$ + Pomnóż$(A[2][2], B[2][2])$
  Zwróć C

Algorytm ten ma złożoność obliczeniową $O(n^3)$. Dokładniej, dla $n = 2^k$ liczba operacji f_k spełnia $f_k = 8f_{k-1} + 4^k$ (osiem mnożeń bloków oraz cztery dodawania bloków $2^{k-1} \times 2^{k-1}$) i $f_0 = 1$.

Dowód. Rozwiązanie równania jednorodnego to $c \cdot 8^k$. Rozwiązania szczególnego szukamy w postaci $a \cdot 4^k$: $a4^k = 8a4^{k-1} + 4^k$, skąd $a = -1$. Z warunku $f_0 = 1$ mamy $c = 2$, zatem $f_k = 2 \cdot 8^k - 4^k$, czyli dla $n = 2^k$: $f(n) = 2n^3 - n^2$. Zgadza się to z danymi dla każdego $n = 1, \ldots, 1000$ (także nieparzystych). □

3.3 Metoda Strassena

Metoda Strassena [1] dokonuje optymalizacji nad metodą Binét'a, zmniejszając liczbę mnożeń bloków z 8 do 7 kosztem zwiększenia liczby dodawań. Dokonywane jest to poprzez wprowadzenie 7 zmiennych pomocniczych $M_1, \ldots, M_7$. Pseudokod algorytmu Strassena wygląda następująco (Algorytm 2):

Algorytm 2 Rekurencyjne mnożenie macierzy metodą Strassena

Procedura POMNÓŻ(A, B) ▷ *A, B: $n \times n$*
  Jeżeli $n = 1$ **wykonaj**
    Zwróć AB
  Jeżeli n jest nieparzyste **wykonaj** ▷ *obieranie, $m = n - 1$*
    wydziel $A[1][1]$ ($m \times m$), kolumnę a_{12}, wiersz a_{21} i liczbę a_{22} (tak samo B)
    $C[1][1] \leftarrow$ POMNÓŻ$(A[1][1], B[1][1]) + a_{12}b_{21}$
    $C[1][2] \leftarrow A[1][1]\, b_{12} + a_{12}b_{22}$
    $C[2][1] \leftarrow a_{21}B[1][1] + a_{22}b_{21}$
    $C[2][2] \leftarrow a_{21}b_{12} + a_{22}b_{22}$
    Zwróć C
  $M_1 \leftarrow$ POMNÓŻ$(A[1][1] + A[2][2],\ B[1][1] + B[2][2])$
  $M_2 \leftarrow$ POMNÓŻ$(A[2][1] + A[2][2],\ B[1][1])$
  $M_3 \leftarrow$ POMNÓŻ$(A[1][1],\ B[1][2] - B[2][2])$
  $M_4 \leftarrow$ POMNÓŻ$(A[2][2],\ B[2][1] - B[1][1])$
  $M_5 \leftarrow$ POMNÓŻ$(A[1][1] + A[1][2],\ B[2][2])$
  $M_6 \leftarrow$ POMNÓŻ$(A[2][1] - A[1][1],\ B[1][1] + B[1][2])$
  $M_7 \leftarrow$ POMNÓŻ$(A[1][2] - A[2][2],\ B[2][1] + B[2][2])$
  $C[1][1] \leftarrow M_1 + M_4 - M_5 + M_7$
  $C[1][2] \leftarrow M_3 + M_5$
  $C[2][1] \leftarrow M_2 + M_4$
  $C[2][2] \leftarrow M_1 - M_2 + M_3 + M_6$
  Zwróć C

Algorytm ten ma złożoność obliczeniową $\approx O(n^{2{,}807})$. Dla $n = 2^k$ liczba operacji spełnia $f_k = 7f_{k-1} + 18 \cdot 4^{k-1}$ (siedem mnożeń bloków, 10 dodawań/odejmowań argumentów i 8 przy składaniu wyniku, każde na blokach $2^{k-1} \times 2^{k-1}$) oraz $f_0 = 1$.

Dowód. Rozwiązanie równania jednorodnego to $c \cdot 7^k$. Rozwiązania szczególnego szukamy w postaci $a \cdot 4^k$: $a4^k = 7a4^{k-1} + 18 \cdot 4^{k-1}$, skąd $a = -6$. Z $f_0 = 1$ mamy $c = 7$, więc $f_k = 7 \cdot 7^k - 6 \cdot 4^k$, czyli dla $n = 2^k$: $f(n) = 7\, n^{\log_2 7} - 6\, n^2$, a stąd $f(n) = O(n^{\log_2 7}) \approx O(n^{2{,}807})$ (sprawdzone dla $k = 0, \ldots, 9$; np. $n = 4$: 247 operacji). Obieranie dla nieparzystego n kosztuje tylko $O(n^2)$ na poziom, więc nie zmienia rzędu złożoności. □

3.4 Metoda Ladermana

W metodzie Ladermana dzielimy A, B i C na 3×3 bloki A_{ij}, B_{ij}, C_{ij} $(i,j = 1,2,3)$ o rozmiarze $h = n/3$. Zamiast 27 iloczynów bloków (jak w podejściu klasycznym) wystarczą 23:

$$
\begin{aligned}
M_1 &= (A_{11}{+}A_{12}{+}A_{13}{-}A_{21}{-}A_{22}{-}A_{32}{-}A_{33})\,B_{22} & M_2 &= (A_{11}{-}A_{21})(B_{22}{-}B_{12}) \\
M_3 &= A_{22}\,(B_{12}{+}B_{21}{+}B_{33}{-}B_{11}{-}B_{22}{-}B_{23}{-}B_{31}) & M_4 &= (A_{21}{+}A_{22}{-}A_{11})(B_{11}{+}B_{22}{-}B_{12}) \\
M_5 &= (A_{21}{+}A_{22})(B_{12}{-}B_{11}) & M_6 &= A_{11}B_{11} \\
M_7 &= (A_{31}{+}A_{32}{-}A_{11})(B_{11}{+}B_{23}{-}B_{13}) & M_8 &= (A_{31}{-}A_{11})(B_{13}{-}B_{23}) \\
M_9 &= (A_{31}{+}A_{32})(B_{13}{-}B_{11}) & M_{10} &= (A_{11}{+}A_{12}{+}A_{13}{-}A_{22}{-}A_{23}{-}A_{31}{-}A_{32})\,B_{23} \\
M_{11} &= A_{32}\,(B_{13}{+}B_{21}{+}B_{32}{-}B_{11}{-}B_{22}{-}B_{23}{-}B_{31}) & M_{12} &= (A_{32}{+}A_{33}{-}A_{13})(B_{22}{+}B_{31}{-}B_{32}) \\
M_{13} &= (A_{13}{-}A_{33})(B_{22}{-}B_{32}) & M_{14} &= A_{13}B_{31} \\
M_{15} &= (A_{32}{+}A_{33})(B_{32}{-}B_{31}) & M_{16} &= (A_{22}{+}A_{23}{-}A_{13})(B_{23}{+}B_{31}{-}B_{33}) \\
M_{17} &= (A_{13}{-}A_{23})(B_{23}{-}B_{33}) & M_{18} &= (A_{22}{+}A_{23})(B_{33}{-}B_{31}) \\
M_{19} &= A_{12}B_{21}, \quad M_{20} = A_{23}B_{32}, \quad M_{21} = A_{21}B_{13} & M_{22} &= A_{31}B_{12}, \quad M_{23} = A_{33}B_{33}
\end{aligned}
\tag{2}
$$

$$
\begin{aligned}
C_{11} &= M_6{+}M_{14}{+}M_{19} & C_{12} &= M_1{+}M_4{+}M_5{+}M_6{+}M_{12}{+}M_{14}{+}M_{15} \\
C_{13} &= M_6{+}M_7{+}M_9{+}M_{10}{+}M_{14}{+}M_{16}{+}M_{18} & C_{21} &= M_2{+}M_3{+}M_4{+}M_6{+}M_{14}{+}M_{16}{+}M_{17} \\
C_{22} &= M_2{+}M_4{+}M_5{+}M_6{+}M_{20} & C_{23} &= M_{14}{+}M_{16}{+}M_{17}{+}M_{18}{+}M_{21} \\
C_{31} &= M_6{+}M_7{+}M_8{+}M_{11}{+}M_{12}{+}M_{13}{+}M_{14} & C_{32} &= M_{12}{+}M_{13}{+}M_{14}{+}M_{15}{+}M_{22} \\
C_{33} &= M_6{+}M_7{+}M_8{+}M_9{+}M_{23}
\end{aligned}
\tag{3}
$$

Na jednym poziomie rekurencji są więc 23 mnożenia bloków oraz $56{+}42 = 98$ dodawań/odejmowań bloków (w każdej sumie pierwszy składnik jest dodatni, więc nie ma osobnej operacji negacji).

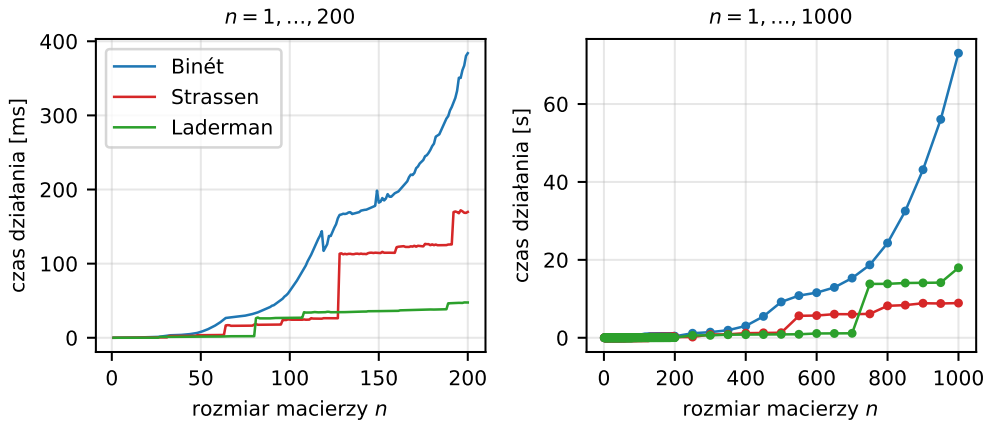
Algorytm 3 Rekurencyjne mnożenie macierzy metodą Ladermana

```
1:  Procedura Pomnóż(A, B)                                   ▷ A, B: n × n
2:      Jeżeli n ≤ 2 wykonaj                                 ▷ 2 × 2 nie da się podzielić na 3 × 3 bloki
3:          Zwróć MnożenieKlasyczne(A, B)
4:      r ← n mod 3
5:      Jeżeli r ≠ 0, m = n − r wykonaj
6:          wydziel A11 (m×m), paski a12 (m×r), a21 (r×m), blok a22 (r×r) (tak samo B)
7:          C11 ← Pomnóż(A11, B11) + a12 b21,   C12 ← A11 b12 + a12 b22
8:          C21 ← a21 B11 + a22 b21,   C22 ← a21 b12 + a22 b22
9:      W przeciwnym razie
10:         podziel A i B na 3 × 3 bloki o rozmiarze h = n/3
11:         Mi ← Pomnóż(·, ·) dla i = 1, . . . , 23 według (2)
12:         Cij ze wzorów (3)
13:     Zwróć C złożone z bloków Cij
```

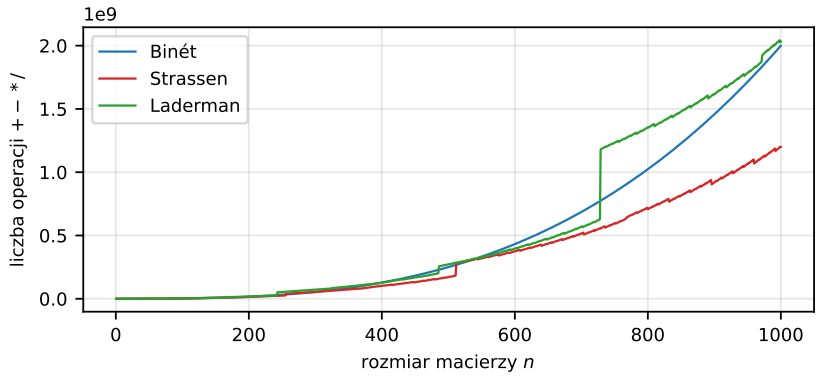
Laderman: $L(n) = 23L(n/3) + 98(n/3)^2$, więc $L(n) = \Theta(n^{\log_3 23}) \approx \Theta(n^{2{,}854})$; dla $n = 3^k$ dokładnie $L(n) = 8\,n^{\log_3 23} - 7\,n^2 = 8 \cdot 23^k - 7 \cdot 9^k$ (sprawdzone dla $k = 0, \ldots, 6$; np. $n = 3$: 121 operacji). Obieranie przy $n \not\equiv 0 \pmod 3$ kosztuje tylko $O(n^2)$, więc nie zmienia rzędu złożoności.

4 Porównanie algorytmów

Dokonaliśmy porównania algorytmów pod względem liczby wykonywanych operacji zmiennoprzecinkowych $(+, -, *, /)$ oraz czasu wykonania, zwiększając rozmiary mnożonych macierzy do $n = 1000$. Rysunek 5 zawiera porównanie czasu działania, a Rysunek 6 liczby operacji wszystkich trzech metod.



Rysunek 5: Czas działania trzech metod rekurencyjnych w zależności od rozmiaru macierzy n



Rysunek 6: Liczba operacji zmiennoprzecinkowych $(+, -, *, /)$ w zależności od n, $n = 1, \ldots, 1000$

4.1 Liczba operacji

Liczba operacji w metodzie Binét'a wynosi dokładnie $2n^3 - n^2$ (dla $n = 1000$: 1 999 000 000), czyli tyle samo co w algorytmie klasycznym. Strassen dla małych n wykonuje *więcej* operacji (1,92× dla $n = 16$, 1,01× dla $n = 100$), bo rekurencja aż do macierzy 1×1 generuje wiele dodawań macierzy; dla $n \geq 536$ ma już zawsze mniej operacji niż Binét (dla $n = 1000$: 1 199 169 602, czyli 60% operacji Binét'a). Wykres Strassena jest „schodkowy": tuż po przekroczeniu potęgi dwójki (np. $n = 511 \to 512$: z $1{,}83 \cdot 10^8$ do $2{,}81 \cdot 10^8$ operacji) rekurencja schodzi o jeden poziom głębiej.

Laderman przy rekurencji do najmniejszych bloków wykonuje *więcej* operacji niż Strassen dla wszystkich $n > 161$ (do 2,27× więcej; mniej tylko w krótkich przedziałach tuż za potęgami dwójki, gdzie Strassen robi skok). Od Binét'a jest tańszy tylko dla 303 spośród $n = 1, \ldots, 1000$ (pierwszy raz dla $n = 219$); dla $n = 1000$ wykonuje 2 032 501 040 operacji, czyli 1,02× tyle co Binét i 1,69× tyle co Strassen. Dla czystych potęg trójki Laderman ma $8 \cdot 23^k$ wobec $2 \cdot 27^k$ u Binét'a, więc liczbą operacji wygrywa z nim dopiero od $n = 3^9 = 19\,683$. Schodki wykresu Ladermana wypadają przy potęgach trójki (np. $n = 728 \to 729$).

4.2 Czas działania

Czas działania (rys. 5) prawie nie zależy od liczby operacji arytmetycznych, tylko od liczby *wywołań rekurencyjnych*. Każde wywołanie kopiuje bloki i alokuje macierze tymczasowe, co kosztuje 70–170 ns, dużo więcej niż pojedyncze + lub ∗. Współczynnik korelacji log(czas) z log(liczba wywołań) dla $n \geq 250$ wynosi 1,00 (Binét), 0,99 (Strassen) i 0,99 (Laderman). Dla $n = 1000$:

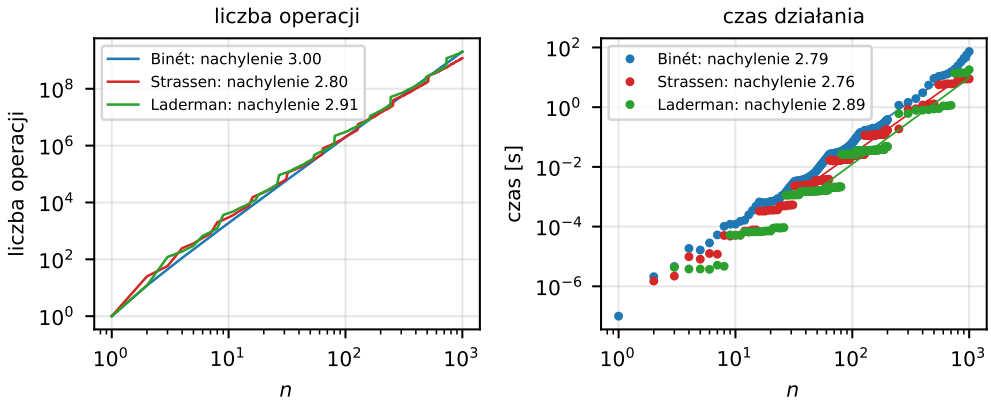
metoda	liczba operacji	liczba wywołań rekurencyjnych	czas
Binét	$1{,}999 \cdot 10^9$	$1{,}08 \cdot 10^9$	73,0 s
Strassen	$1{,}199 \cdot 10^9$	$5{,}38 \cdot 10^7$	8,9 s
Laderman	$2{,}033 \cdot 10^9$	$1{,}61 \cdot 10^8$	18,0 s

Binét i Laderman wykonują prawie tyle samo operacji, ale Laderman jest 4 razy szybszy, bo dzieli macierz przez 3, a obierane paski mnoży jednym wywołaniem klasycznym. Ma więc mniej poziomów rekurencji i mniej wywołań. Najlepiej widać to przy skoku $n = 700 \to 750$: liczba wywołań Ladermana rośnie z $7{,}0 \cdot 10^6$ do $1{,}55 \cdot 10^8$ (o jeden poziom rekurencji więcej), a czas z 1,15 s do 13,8 s. Dla $n \leq 700$ Laderman jest najszybszy w większości punktów pomiarowych (np. dla $n = 200$: 48 ms wobec 170 ms Strassena i 384 ms Binét'a), mimo że wykonuje najwięcej operacji. Wyjątkiem są przedziały tuż za potęgami trójki (np. $n = 81, \ldots, 127$ i $n = 250$), gdzie wygrywa Strassen.

4.3 Oszacowanie złożoności obliczeniowej

Teoretyczne złożoności wyprowadzono w rozdziale 3: $O(n^3)$ dla Binét'a, $O(n^{\log_2 7}) \approx O(n^{2{,}807})$ dla Strassena i $O(n^{\log_3 23}) \approx O(n^{2{,}854})$ dla Ladermana. Sprawdzamy je eksperymentalnie (rys. 7): dopasowanie prostej do danych w skali log-log daje wykładnik α w zależności $\sim n^\alpha$.

Dla liczby operacji ($n \geq 16$, wszystkie n): Binét $\alpha = 3{,}00$, Strassen $\alpha = 2{,}80$, Laderman $\alpha = 2{,}91$. Dla samych $n = 2^k$ Strassen daje 2,87, a dla samych $n = 3^k$ Laderman daje 2,88. To nieco więcej niż teoretyczne 2,807 i 2,854, bo dla małych n ważne są składniki $-6n^2$ i $-7n^2$. Dla czasu ($n \geq 50$): Binét $\alpha = 2{,}79$, Strassen $\alpha = 2{,}76$, Laderman $\alpha = 2{,}89$. Dla Binét'a przy $n \geq 250$ wychodzi 2,98, zgodnie z teorią. U Strassena i Ladermana czas rośnie schodkowo (skoki przy potęgach 2 i 3), więc nachylenie wyznaczone z czasu silnie zależy od wybranego zakresu n. Wiarygodną miarą złożoności jest tu liczba operacji, a nie czas.



Rysunek 7: Wykresy w skali log-log; nachylenie dopasowanej prostej oznacza wykładnik α w zależności $\sim n^\alpha$

Wniosek: Binét ma złożoność $O(n^3)$, Strassen $O(n^{\log_2 7}) \approx O(n^{2{,}81})$, a Laderman $O(n^{\log_3 23}) \approx O(n^{2{,}85})$. Asymptotycznie Laderman jest więc lepszy od Binét'a, ale gorszy od Strassena. Przy rekurencji do najmniejszych bloków przewaga Strassena nad Binét'em ujawnia się dla n rzędu kilkuset, a Ladermana dopiero dla n rzędu $2 \cdot 10^4$.

4.4 Weryfikacja poprawności: przykład 4×4

Do porównania z Octave/MATLAB wybrano $A =$ `magic(4)`, $B = A^T/7$ (te same macierze są zaszyte w trybie `--example` wszystkich programów). U Ladermana $n = 4$ daje jedno obieranie ($r = 1$) i jeden krok właściwego algorytmu na rdzeniu 3×3. Skrypt `compare.m` wczytuje wyniki programów C++ i porównuje je z $A * B$ w Octave/MATLAB. Wynik referencyjny:

$$C_{\text{ref}} = AB \approx \begin{pmatrix} 62{,}5714 & 33{,}7143 & 47{,}4286 & 21{,}4286 \\ 33{,}7143 & 44{,}2857 & 39{,}7143 & 47{,}4286 \\ 47{,}4286 & 39{,}7143 & 44{,}2857 & 33{,}7143 \\ 21{,}4286 & 47{,}4286 & 33{,}7143 & 62{,}5714 \end{pmatrix}.$$

metoda	liczba operacji	$\max \lvert C - C_{\text{ref}} \rvert$
Binét	112 $(= 2 \cdot 4^3 - 4^2)$	$7{,}1 \cdot 10^{-15}$
Strassen	247 $(= 7 \cdot 7^2 - 6 \cdot 4^2)$	$1{,}4 \cdot 10^{-14}$
Laderman	188 $(= 121 + 67)$	$7{,}1 \cdot 10^{-15}$

Wszystkie metody dają wynik zgodny z mnożeniem wbudowanym z dokładnością do błędów zaokrągleń (rzędu 10^{-14}); Strassen ma nieco większy błąd, bo wykonuje więcej dodawań/odejmowań w innej kolejności. U Ladermana 121 operacji $(= 8 \cdot 23 - 7 \cdot 9)$ przypada na rdzeń 3×3, a 67 na cienkie iloczyny przy obieraniu.

Literatura

[1] V. Strassen, *Gaussian elimination is not optimal*, Numerische Mathematik 13 (1969), s. 354–356.

[2] S. Huss-Lederman, E. M. Jacobson, J. R. Johnson, A. Tsao, T. Turnbull, *Implementation of Strassen's algorithm for matrix multiplication*, Supercomputing '96 (1996).